

Golden Thread Plugin

Golden Thread Plugin — Documentation

Reader: quick lookup, and the printed PDF **Claims last checked against the code:**
2026-09-17 — see *The documents, and what belongs in each* in [CLAUDE.md](#) .

Version gt 0.16.3 / gt-wiki 0.2.2 / gt-demo, gt-watch, gt-report-card, gt-farm, gt-flow 0.16.3

Golden Thread turns an Obsidian vault into the single source of truth for all AI memory across every project and every session. The tiered rule model introduced in v0.6.0 now carries **ten hook-backed Core rules** enforced at three points in the turn, 0.9.12 added `gt-route` for the middle of a session, and 0.9.13 makes the session-start component check verify that the hooks are **wired**, not merely installed, 0.11.0 makes `log.md` and `decisions.md` generated files so concurrent sessions cannot overwrite one another, 0.12.3 stops a vault tool running against a vault it was never told to touch, 0.12.4 makes parallel execution the default for divisible work in every project, 0.12.5 stops code being committed before its tests have been seen to pass, 0.13.0 makes an upgrade from any older release end where a fresh install would, 0.14.0 splits optional parts into modules you can decline, and 0.15.0 makes six of them — gt-wiki 0.2.1 (an LLM-powered knowledge base with immutable sources and interlinked pages), gt-demo (the guided tour), gt-watch (upstream repos), gt-report-card (the session report card), gt-farm (work packets for an external AI) and the new gt-flow, which draws the vault's event stream as a timeline of knowledge climbing the ladder. 0.16.0 gave contributed **packs** — plain JSON data, reviewed and merged, never third-party code — somewhere to come from and something to read them, shipping `gt-scan` , `gt-optimize` , `gt-handoff` , `gt-allin` and `gt-allin-commit` ; 0.16.1 added `gt-context` (the first consumer of the registry's model-reachable tier) and `gt-validation` (a verification receipt that goes stale when the file it describes changes); and 0.16.2 re-asserts the Core rules after a compaction as well as on every prompt, lets a module hook declare a `timeout` , and refuses a copyleft licence by its current SPDX spelling; 0.16.3 makes `install.sh`

refuse when MANIFEST.json is tracked but missing rather than regenerating one, and stops `gt_tasks.py` overwriting TASKS.md in silence.

What It Does

Instead of scattered `.claude/memory/` files and CLAUDE.md snippets that live and die per session, Golden Thread gives every fact a permanent home in a structured vault. Knowledge flows up a hierarchy from session notes into the vault, and the right facts are always in scope when you need them.

```
session conversation
  ↓ gt-work / gt-promote
project memory (memory/*.md)
  ↓ gt-promote
project files (research.md / decisions.md / design.md / spec.md / runbook.md)
  ↓ gt-promote
Knowledge/ (cross-project wiki pages, backed by immutable Sources/)
  ↓ gt-promote
global-memory/ (loaded in every session, all projects)
```

Facts move up the hierarchy as they prove themselves general. They never move back down and are never silently deleted.

Core Rules (gt 0.16.3)

Golden Thread defines a tiered rule model that separates rules by scope and enforcement strength. **Ten Core rules ship as of 0.15.0** and remain ten in 0.16.3, up from one at 0.6.0 (the count read "seven" from 0.9.10 through 0.12.3, one behind the files):

#	Rule
1	Register your session and claim a vault file before writing it; never write a file another live session has claimed.

#	Rule
2	Name the vault on every mutating tool run — <code>--vault</code> or <code>--dry-run</code> — never let the target be inferred.
3	Never put a secret's value into the session — not to inspect it, not to redact it, not to check it.
4	Run the tests before you commit code, and say which ran — never commit code whose tests you have not seen pass.
5	Begin every response with the current wall-clock timestamp — before any other text you emit.
6	<code>global-memory/</code> contains only facts needed in EVERY project.
7	Do not auto-load the full memory index.
8	Parallelise any work that can be parallelised, in every project — independent units run concurrently within one machine-wide budget shared by every session, and serial execution must be justified, not assumed. (Reworded in 0.15.0: the <code>parallel_max</code> budget is one for the whole machine, not one per session.)
9	A secret's value rests only in the secrets store or a mode-600 file the store wrote — never in source, a vault file, a repo, a log, or a session.
10	Label every derived figure you present as fact with its verification state — <code>unverified</code> , <code>self-verified</code> , or <code>independently verified</code> .

The numbers are the order the `UserPromptSubmit` hook injects them in. Verify the set at any time with `echo '{}' | ~/.claude/golden-thread/hooks/inject_core_rules.sh`.

Since 0.16.2 the same script is registered a **second time**, on `SessionStart` with a `compact` matcher, because a compaction does not preserve what a hook added earlier: `project-root`, `CLAUDE.md` and auto memory are re-injected from disk, while hook-added context is summarized with the rest of the conversation. The exposure this closes is narrow and worth stating precisely — `UserPromptSubmit` has no compaction exception, so the next user prompt always restored the rules verbatim; what was exposed was the *remainder of the turn* in which an auto-compaction fired. The mechanical tier was never affected: the `PreToolUse` guards and the `Stop` validator are event-driven commands, not context. What lapsed was the re-assertion, not the backstop.

Three scope levels:

Level	Meaning
Core	Applies to every chat, every project, no exceptions. Un-removable.
Context	Applies to this project or session type only.
Generic	Default; no special standing.

Two enforcement strengths:

Strength	Meaning
Reminder	Injected on every turn via <code>UserPromptSubmit</code> hook.
Validated	Checked by a <code>Stop</code> hook that blocks any reply violating it; machine-enforced.

Since 0.12.3 a **second `PreToolUse` guard** (`guard_vault_writes.sh`) denies a vault-mutating tool run that does not say *which* vault it means — no `--vault` , no `--dry-run` , no `GT_VAULT` . It exists because a session rehearsing the 0.11.0 migration in a scratch copy had one tool that took `--vault` and one that did not, so the same loop migrated all 42 `decisions.md` files of the live vault. A rehearsal that cannot be told where to rehearse is not a rehearsal. Read-only subcommands are never denied, and anything it cannot parse with certainty is allowed — it fails open, as every guard here does, because a guard that blocks wrongly makes every session unusable.

Since 0.9.5 a third point exists: a **`PreToolUse` hook** (`guard_session_claims.sh`) denies a `Write / Edit` to a vault file another live session holds a claim on. It fails open on any parse failure, and deliberately does not cover `Bash` writes — parsing arbitrary shell fails open so often it would train the guard to be ignored.

Core/Validated rules are hook-backed. Hook scripts install to `~/.claude/golden-thread/hooks/` during `install.sh` . `settings.json` references them by absolute path so they survive vault renames and project moves. `gt_paths.py` is a self-healing resolver — if a recorded path is stale, it locates `core-rules/` by scanning the vault rather than failing silently.

The canonical rule definitions live in `Projects/golden-thread/core-rules/` inside the vault. Editing a rule file there changes what the hook injects — the rule text is never duplicated into the script.

The canary: `core_timestamp_every_message.md` is designated Core/Validated not because a missing timestamp causes real harm, but because it is trivially observable at the top of every reply. When

the timestamp disappears, enforcement has broken.

gt Skills (23)

Setup

Command	What it does
<code>/gt:gt-init</code>	Create a new vault from scratch, or wire an existing vault to a project. Idempotent — safe to re-run.
<code>/gt:gt-create</code>	Scaffold a new project folder with the standard structure. Captures your brain dump into <code>idea.md</code> . Supports sub-projects, tags, and runbook creation.

Daily Work

Command	What it does
<code>/gt:gt-open</code>	Load a project at session start. Reads the core project docs in order — <code>source.md</code> first, so you know which host serves which role before touching code, then <code>idea</code> → <code>research</code> → <code>decisions</code> → <code>design</code> — and <i>indexes</i> the memory files rather than loading them. Summarizes state and asks where to pick up.
<code>/gt:gt-route</code>	Mid-session. Names what the session has actually become, says where its output belongs, and checks you are in the right project, harness and model. For when a session drifted from what it opened with, or you cannot name what you are doing.
<code>/gt:gt-work</code>	Write back session findings. Appends to <code>research.md</code> , adds ADRs to <code>decisions.md</code> , refines <code>design.md</code> , creates <code>spec.md</code> when design is complete, and flags content for <code>PROTOCOL.md</code> .
<code>/gt:gt-ingest</code>	Bulk-import an existing project's memory files, CLAUDE.md rules, and notes into the vault. External sources are stored immutably in <code>Sources/</code> before being synthesized into Knowledge pages.

Command	What it does
<code>/gt:gt-review</code>	Empty the inbox: sweep <code>INBOX.md</code> — plus Obsidian daily notes if you keep them — for captured-but-unfiled thoughts and route each one into a tracked project.

Knowledge Management

Command	What it does
<code>/gt:gt-query</code>	Look something up — reads <code>index.md</code> first, follows wikilinks, falls back to grep. The vault is always checked before the web.
<code>/gt:gt-promote</code>	Graduate a fact up the hierarchy: project memory → project files → Knowledge wiki page → global-memory. Also handles new project scaffolding and retiring stale content.
<code>/gt:gt-refresh</code>	Check <code>Sources/</code> for upstream changes. Supersedes outdated sources with new immutable files — never edits the old one. Updates Knowledge pages that cited the changed source.

Context & Verification

Command	What it does
<code>/gt:gt-validate</code>	Verify a claim by re-deriving it with a fresh-context validator — never by reviewing the reasoning that produced it. Use before recording a finding as fact or before a production change.

Maintenance

Command	What it does
<code>/gt:gt-upgrade</code>	Bring an existing vault up to the installed release: run the migrations it has not had, take the release's changes into <code>PROTOCOL.md</code> and <code>CONVENTIONS.md</code> without losing local edits, add newly shipped Core rules, stamp the vault. Rehearse with <code>--dry-run</code> ; it refuses a dirty tree and backs up before applying.

Command	What it does
<code>/gt:gt-doctor</code>	One report for the whole install: plugin version, component drift, hook wiring, pending vault migrations, stray workers, unpushed commits, publish-destination drift and a lint summary. Exit 2 means a check <i>could not run</i> , which is deliberately distinct from clean.
<code>/gt:gt-lint</code>	Audit the vault for structural problems: 18 checks covering broken wikilinks, orphaned pages, missing index entries, unlisted memory files, Knowledge pages citing superseded sources, stale pages, and <code>core-unenforced</code> — a Core rule that is stored but wired to no hook.
<code>/gt:gt-optimize</code>	Find vault content that costs context and earns nothing back: a fact duplicated across memory files, an index row pointing at a file that is gone, a <code>global-memory/</code> file over budget, a relative date in a file that will be read months later. Splits findings into SAFE (applied with <code>--apply</code>) and JUDGEMENT (reported only — deleting knowledge is not reversible by reading a diff). Never writes <code>core-rules/</code> , and needs a flag for <code>global-memory/</code> .
<code>/gt:gt-scan</code>	Scan code against the language definitions in effect on this machine — naming conventions and encoding, per language, all of it from packs rather than from the script (fourteen languages ship with definitions; <code>gt_scan_language.py --languages</code> lists what is in effect here). An aggregator over leaf scanners: it reports how many members RAN alongside what they found, because "nothing is wrong" and "nothing was checked" otherwise print identically.
<code>/gt:gt-allin</code>	Run every check in one command — scan, lint, the optimize report, install health — and report how many members actually RAN alongside what they found, because a short finding list from a half-failed run reads exactly like a clean bill of health. Never pushes and never applies a change: <code>--suggest-push</code> prints the command for you, and refuses even that when anything failed.
<code>/gt:gt-context</code>	Render the definitions this vault marks as model-reachable — <code>vocabulary</code> , <code>validation_rules</code> , <code>runbook</code> — inside an explicit untrusted-data envelope, hard-capped, with every line naming the pack and tier it came from. The envelope is not a security control — no in-context framing is: <i>Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents</i> (NAACL 2025 Findings, arXiv:2503.00061) attacked eight published defences and broke all eight, above 50% attack success in every case. What the envelope provides is <i>identifiability</i> : it marks content as someone's definition rather than

Command	What it does
	as the system speaking. What protects a session is that packs are reviewed before merge, and that a pack can only reach a session already trusting the vault. Asking for a Tier A slot is a usage error, not an empty section.
<code>/gt:gt-validation</code>	Record what a validation established about a file — what was verified AND what it could not determine — stamped with the file's content hash, so the recorded definition goes visibly stale the moment the file changes. A file with no receipt reports as <i>unknown</i> , deliberately distinct from clean.
<code>/gt:gt-allin-commit</code>	Commit, but only once the checks pass and a passing test receipt covers every staged file. Refuses on <code>main / master</code> without a flag, refuses when a check <i>could not run</i> (an unknown is not a finding anyone can accept), and never pushes — a commit is reversible with <code>git reset</code> , a push is fetched by other people. It checks the receipt itself because <code>guard_test_before_commit</code> is a PreToolUse hook and cannot see a script running git.
<code>/gt:gt-handoff</code>	Write the next session a handoff it can trust: the facts gathered from the project and the repository, each labelled with its source and whether it was actually verified. The design narrative is deliberately left for the session that did the work — a script that invents it produces a document that reads finished and is not.
<code>/gt:gt-runbook-lint</code>	Scan all project <code>runbook.md</code> files for content that has drifted into multiple runbooks. Routes duplicated content to the right shared layer via <code>gt-promote</code> .
<code>/gt:gt-settings</code>	View and change what Golden Thread does on its own. gt's own settings are <code>component_updates</code> , <code>version_check</code> , <code>orphan_check</code> , <code>push_check</code> , <code>test_gate</code> , <code>parallel_work</code> , <code>parallel_max</code> and <code>protected_paths</code> ; each installed module registers its own from <code>module.json</code> and they are listed under the module's name (<code>report_card</code> , <code>closeout_check</code> , <code>watch</code>). Every automatic behaviour is registered here and every one can be switched off.

Modules (6)

Optional parts, each a separate plugin in the `golden-thread-plugin` marketplace, declared by a `module.json`, versioned with `gt` and installed by `install.sh` while on. `bash install.sh --list-modules` shows each one's state; `--without <name>` / `--with <name>` change it and the choice is remembered. A module's hooks are wired only while it is on and are always *reporter* hooks — a Core-rule enforcement hook can never belong to a module. A module hook may declare an optional `timeout` in whole seconds (1–600); `report-card` declares 15 on `SessionEnd`, because `SessionEnd` hooks share a 1.5-second budget by default and one that overruns is cancelled with its **output discarded**, so a report card could have started truncating invisibly. Module settings (with an optional long `detail`) register from `module.json` and are listed under the module's name in `/gt:gt-settings`; a value set for a module that is later switched off is kept.

Command	Module (plugin) · default	What it does
<code>/gt-wiki:gt-wiki</code> and four more	<code>wiki (gt-wiki) · on</code>	A standalone LLM wiki: immutable sources, interlinked pages, ingest, lint and refresh — see below.
<code>/gt-demo:gt-demo</code>	<code>demo (gt-demo) · on</code>	A guided tour on the fictional PizzaBot 3000 project, run in its own throwaway vault: nine core acts (Core rules, open, task rollup, capture, route, validate, lint, promote, inbox and close) plus one act from each installed module that ships one — <code>wiki</code> , <code>watch</code> and <code>flow</code> , so twelve with the defaults. <code>start</code> , <code>tour</code> , <code>end</code> , <code>clean</code> , <code>remove</code> , <code>status</code> .
<code>/gt-watch:gt-watch</code>	<code>watch (gt-watch) · on</code>	Watch any git repo. <code>add <url></code> writes a watch note; a cron <code>fetch (gt_watch.py fetch)</code> classifies every change by rules — P0 for security advisories, CVE/GHSA ids or security releases; review for new releases, major bumps, changes under <code>watch_paths</code> ; routine otherwise — and a <code>SessionStart</code> hook reports unacknowledged changes, P0 first. <code>show</code> explains a change, <code>ack</code> marks it seen. Setting <code>watch</code> , default <code>off</code> . <code>--without watch</code> also removes its crontab line. Was <code>/gt:gt-watch</code> .
<i>(no command)</i>	<code>report-card (gt-report-card) · on</code>	The session report card at <code>/compact</code> and session end — saved as a notice and shown at the next session start, because output at those events is never displayed — and

Command	Module (plugin) · default	What it does
		the project close-out question. Settings <code>report_card</code> , <code>closeout_check</code> .
<code>/gt-farm:gt-farm</code>	<code>farm (gt-farm) · off</code> fresh, on for upgraders	Route bulk, mechanical, or second-opinion tasks to an external AI service as a self-contained work packet. All four gates (Stateless, Self-contained, Checkable, Releasable) must pass before a task leaves. Results come back unverified. Was <code>/gt:gt-farm</code> ; a machine upgrading from a gt that shipped it keeps it on.
<code>/gt-flow:gt-flow</code>	<code>flow (gt-flow) · on</code>	New in 0.15.0. Renders <code>events.jsonl</code> as one offline HTML file: a lane per project, time left to right, an arrow each time an item climbed a level. <code>--redact</code> hashes every name before the page is shared; task events are hidden until <code>--tasks</code> or a click; <code>--project</code> , <code>--since</code> . Never writes the vault.

Movement events (0.15.0)

`gt_events.py` (schema v1, since 0.13.0) now receives events from the operations themselves: `gt_log.py add ... --event <kind> --item <path>` spools a log line and its event in one command (used by `gt-promote` and `gt-refresh`); `gt-review` , `gt-work` and `gt-ingest` call `gt_events.py emit` ; `gt_adr.py allocate` emits `adr` ; `vault_init.py` emits `create` , `rename` , `merge` and `archive` (the only source of `archive`); `gt_tasks.py` emits `task.open` / `task.done` once the stream has been seeded. An event can never fail the operation it records. A vault older than the events recovers its history with `gt_events.py --vault <vault> backfill --dry-run` (then without `--dry-run`): git and `log.md` rebuilt as `actor: backfill` events, idempotent.

Concurrent sessions stop colliding (0.12.3)

`log.md` and `decisions.md` are the two files every session appends and none owns. In one working tree there are no branches to collide and no merge to resolve — just last-writer-wins, invisible when it happens.

Both are now **generated**, joining `TASKS.md` :

File	Written by	Rendered by
<code>log.md</code>	<code>gt_log.py add</code> — one spool file per session	<code>gt_log.py merge</code>
<code>decisions.md</code>	<code>gt_adr.py allocate <project></code> — one file per ADR	<code>gt_adr.py merge <project></code>

No session writes a shared file, so there is nothing to claim and nothing to guard, and `git add` scopes to your own work by construction rather than by discipline.

ADR numbers are the harder half, because a number is a scarce identity — two sessions may both write a log line in the same instant, but they cannot both be ADR-6. A counter that is read and then written back does not fix that: read-then-write is two operations, so both sessions read 5, both write 6, and both use ADR-6. `gt_adr.py allocate` instead creates the slot with a single atomic exclusive create, so the filesystem decides the winner. Verified under contention: 100 racing process pairs, 200 allocations, no duplicates.

Migration (`gt_log.py migrate` , `gt_adr.py migrate <project>`) freezes the existing file as a baseline that sorts first — no history is rewritten and nothing is renumbered — and refuses unless the merge reproduces the original byte for byte. It also refuses on a project whose ADR numbers already collide, because renumbering would invalidate inbound references; `gt-lint 's adr-collision` check reports those for an owner to resolve.

One limit, stated plainly: an exclusive create is atomic on one filesystem. Two machines writing one project through a synced folder can both take a slot, and the sync resolves it as a conflicted copy. The lint check is what catches that; this is not distributed consensus.

Installed, and actually wired (0.9.13)

The session-start component check answers two questions, not one:

Question	How
Are the right files here, unmodified?	every shipped file is hashed into <code>MANIFEST.json</code> at install time and compared
Is any of it connected to anything?	<code>MANIFEST.json</code> also declares the hook entries the plugin expects in <code>settings.json</code> — eleven as of 0.16.2 — and the check compares those too

Before 0.9.13 only the first question was asked, so a machine with every file installed and an empty `settings.json` reported **components: clean** while nothing ran. The clean line now reads *"installed matches , all 11 hooks wired"* — the second clause is the part that was missing.

Two states are reported: `unwired` (no entry for that event names the script — it never runs) and `badpath` (wired, but the command names a path that does not exist on this machine, which is what a deleted version directory or an unquoted path containing a space produces).

The declaration lives in `gt_components.HOOK_REGISTRATIONS` and is what `install.sh` registers *from*, so the installer and the checker cannot disagree about what "wired" means. `install.sh` verifies its own five entries immediately after writing them; the six enforcement hooks are owned by `vault_init.py install-core-rules`, which needs a vault. `selftest.sh` asserts every declared entry from outside — the only vantage point that still works when nothing is wired at all.

gt-wiki Skills (5)

The gt-wiki plugin provides an LLM-powered knowledge base separate from the Golden Thread memory hierarchy. Sources are ingested immutably; Knowledge pages are synthesized summaries linked bidirectionally.

Key design decision: The vault `CLAUDE.md` is kept slim (architecture + pointer only). Page format schema lives in `Knowledge/_template.md` and is read on demand by the ingest and refresh skills — it is not loaded into every session's context.

Command	What it does
<code>/gt-wiki:gt-wiki-init</code>	Set up a new wiki vault from scratch. Runs <code>vault_init.py</code> deterministically — creates <code>Sources/</code> , <code>Knowledge/</code> , <code>index.md</code> , <code>log.md</code> , seeds <code>CLAUDE.md</code> from template, and stamps <code>Knowledge/_template.md</code> .
<code>/gt-wiki:gt-wiki</code>	Query the wiki. Reads <code>index.md</code> first, follows wikilinks, falls back to grep, deep-digs Sources for precision. Logs every query.
<code>/gt-wiki:gt-wiki-ingest</code>	Add a source: fetch or paste, store immutably in <code>Sources/</code> , discuss with user, write Knowledge pages per <code>_template.md</code> , cross-link bidirectionally, then update <code>index.md</code> and <code>log.md</code> through <code>wiki_log.py</code> so every entry has the same shape. Records <code>upstream_sha</code> for sources inside a git repo.

Command	What it does
<code>/gt-wiki:gt-wiki-lint</code>	Run <code>wiki_lint.py</code> (10 deterministic checks). Interprets findings, proposes fixes, records declines in <code>lint-declines.md</code> so nothing gets re-litigated.
<code>/gt-wiki:gt-wiki-refresh</code>	Check selected sources for upstream changes. <code>wiki_refresh.py</code> detects changes deterministically for local sources (<code>git fetch</code> + <code>git diff</code> from the source's <code>upstream_sha:</code> , or its <code>ingested:</code> date) and flags web-only sources for the LLM to fetch and compare. Supersedes changed ones with new immutable source files. Updates citing Knowledge pages.

Vault Structure

<code><vault>/</code>	
<code>CLAUDE.md</code>	← architecture + pointer to Knowledge/_template.md
<code>index.md</code>	← navigational index of all Knowledge pages
<code>log.md</code>	← audit trail: GENERATED from the per-session spools
<code>INBOX.md</code>	← the capture point: one checkbox line, any session
<code>TASKS.md</code>	← GENERATED cross-project task rollup (<code>gt_tasks.py</code>)
<code>review-queue.md</code>	← items flagged for owner review
<code>lint-declines.md</code>	← lint findings you declined, so they are not re-litigated
<code>Sources/</code>	← IMMUTABLE raw originals
<code>YYYY-MM-DD <title>.md</code>	
<code>Knowledge/</code>	← cross-project wiki pages
<code>_template.md</code>	← page format schema (read by ingest/refresh skills)
<code><Page Title>.md</code>	
<code>global-memory/</code>	
<code>MEMORY.md</code>	← index; loaded every session
<code><topic>.md</code>	
<code>Projects/</code>	
<code>README.md</code>	← master project list
<code>CONVENTIONS.md</code>	← lifecycle phases, file roles, tag taxonomy
<code>PROTOCOL.md</code>	← cross-project process rules
<code>INFRASTRUCTURE.md</code>	← server fleet, defined once
<code><project-slug>/</code>	

```

README.md      ← status board + YAML frontmatter
source.md     ← where code lives, topology, deploy file plan
idea.md       ← original brain dump – IMMUTABLE
research.md   ← append-only findings
decisions.md  ← append-only ADRs
design.md      ← current architecture
spec.md       ← handoff artifact (created when design is complete)
runbook.md    ← operational procedures (optional)
memory/
  MEMORY.md   ← session memory index
  <topic>.md

golden-thread/
  core-rules/ ← canonical Core rule definitions
  tools/      ← gt_log.py, gt_adr.py, gt_spool.py, gt_tasks.py,
               gt_session.py, gt_closeout.py, gt_edits.py,
               gt_events.py, safe_write.py

  spool/      ← per-session log and ADR spools (log.md/decisions.md
               are rendered from these, never written directly)

```

Immutability Model

File	Mutability	Rule
Sources/*.md	Immutable	Never edited; superseded by new files
idea.md	Immutable	Origin story; never changed
research.md	Append-only	History is extended, never rewritten
decisions.md	Append-only	Reversed by adding a new ADR, never by editing
design.md	Mutable	Always describes the current architecture
spec.md	Mutable (scope only)	Self-contained handoff doc
runbook.md	Mutable	Operational HOW-TO; incubator before CLAUDE.md

File	Mutability	Rule
<code>PROTOCOL.md</code>	Append-only	Cross-project process rules
<code>Knowledge/*.md</code>	Mutable	Synthesized summaries; cite <code>Sources/</code>
<code>core-rules/*.md</code>	Mutable	Editing here changes what hooks inject

Context Footprint

Unverified — undated. The token figures below carry no date and no provenance, and nothing in the source tree derives or checks them; the 2026-09-17 accuracy pass could not establish which release they were measured against. Read them as orders of magnitude, not as measurements, and re-measure before quoting them anywhere.

What loads	When	Approx tokens
<code>~/.claude/CLAUDE.md</code> (global instructions)	Every session	~200
<code><vault>/CLAUDE.md</code> (architecture + pointer)	Every session in vault dir	~100
<code>global-memory/MEMORY.md</code> (index only)	Every session	~50
<code>Knowledge/_template.md</code> (page schema)	Only during ingest/refresh	~150
Project memory files	On demand via <code>gt-open</code>	Varies

The slim CLAUDE.md design (introduced in gt-wiki 0.1.0) drops the vault's cold-start footprint from ~1,300 tokens to ~350.

Source Topology

Every project records where its code lives in `source.md`.

Topology	Meaning
<code>local</code>	A folder on this machine. No deploy step.
<code>remote</code>	One repo, one server; server pulls from GitHub.
<code>bastion-jump</code>	Several servers, all reached through one gateway host.
<code>bastion-direct</code>	Several servers, each reachable independently.

Bastion projects carry a **file plan** marking each deployed file `static` (byte-identical everywhere) or `unique` (per-server variant that must never be cross-deployed).

Install

```
# From zip
unzip golden-thread-plugin.zip
cd golden-thread-plugin
bash install.sh
# Restart Claude Code
```

Installs `gt` (v0.16.2) and each module that is on — `wiki`, `demo`, `watch`, `report-card` and `flow` on by default, `farm` off for a fresh install — as separate plugins under the `golden-thread-plugin` marketplace. Choose modules with `--list-modules`, `--without <name>` and `--with <name>` (remembered). Re-running upgrades from any older release. Requires Python 3.8+.

`install.sh` installs the **newest version directory** present, not a hardcoded constant — pass an argument only to roll back deliberately (`./install.sh 0.14.0`). Never pipe it to `head : set -o pipefail` turns the closed pipe into an abort partway through, leaving the cache updated and registration undone.

Typical Session Pattern

```
/gt:gt-open my-project      # load project, summarize state

# ... work happens ...

/gt:gt-route                # "where is this going?" – run it when the session
                            # has drifted, or before writing something down and
                            # you are unsure which file it belongs in

/gt:gt-work                 # write back findings, update docs

# periodically
/gt:gt-lint                 # catch structural drift
/gt-wiki:gt-wiki-ingest <url> # add a source to the wiki
/gt:gt-promote              # graduate a finding to Knowledge or global-memory
/gt-flow:gt-flow            # see how knowledge moved (--redact before sharing)
```

Requirements

- Python 3.8+
- Claude Code (any version)
- Obsidian (optional — the vault is plain markdown; Obsidian is the GUI layer)